Pack 07 · Profiling and the GPU

How to find where the time goes before changing anything: the measure-floor-locate loop, memory snapshots, Nsight Systems and Nsight Compute, and why fusion is a memory fix. Lessons 55–58, 60, 61. Answer each question out loud in about a minute, then check the card.

Q1 · "Just turn on torch.compile" PRIORITY

Lessons: 55, 61

> **INTERVIEWER**
>
> Users say decode on our Llama 3 8B endpoint (one H100, BF16) feels slow. The manager says "just turn on torch.compile." Before changing anything, what do you measure, what floor do you compare against, and how do you decide whether compute, memory or overhead owns the missing time?

60-second answer

- **Measure one clean number first:** the time per output token. I call a GPU synchronize before stopping the timer, throw away the warm-up runs, and take the median of several runs, so the number reflects steady decode and not noise.
- **Compare it with a floor:** every decode step has to read all the weights, so the fastest a step can be is weight bytes ÷ bandwidth = 16 GB ÷ 3.35 TB/s ≈ **4.8 ms**. **MBU** is that floor divided by the measured time. If we measure 25 ms per token, MBU is 4.8 ÷ 25 ≈ **19%**, which means most of the time is going somewhere other than reading weights.
- **Locate the missing time with a trace:** if the kernels are sparse with gaps between them, the GPU is waiting on CPU launches, so it's **overhead**. If the kernels are packed back to back and MBU is 70–80%, it's **memory**. If they're packed but MBU is still low, then **one slow kernel** owns the time, and I take that kernel to Nsight Compute.
- **What torch.compile actually fixes:** `mode="reduce-overhead"` adds CUDA graphs plus fusion, so it fixes **overhead** only. On the lesson's 7B Llama (on an A100) it took decode from 25.5 to 107 tokens/s, which moved MBU from 17% to 72%. If decode is already memory-bound, only **fewer bytes** helps: int8 weights reached 157 tokens/s because reading fewer bytes per step raises the floor itself.
- **Change one thing, re-measure, and log it**, because the bottleneck moves after every fix.

Key numbers: floor = weight bytes ÷ bandwidth; MBU = floor ÷ measured time.

Likely follow-up: "MBU is 75% after compile. Next?" At 75% the step is close to the bandwidth limit, so decode is now memory-bound and the remaining levers are about bytes per token. I'd either quantize the weights so each step reads fewer bytes, or batch more users onto each weight read so one read produces more tokens.

> **TRAP**
>
> I stopped at "decode should be memory-bound" without computing MBU, and couldn't say which trace pattern means overhead versus memory, or which one torch.compile actually fixes.

Q2 · OOM at hour 6 with 60 GB allocated PRIORITY

Lessons: 56

> **INTERVIEWER**
>
> Our server runs fine for hours, then dies with CUDA out-of-memory around hour 6. At the crash, `torch.cuda.memory_allocated()` says 60 GB on an 80 GB card. What could the memory snapshot show, and how can it be out of memory with 20 GB free?

60-second answer

- **How I'd capture it:** call `torch.cuda.memory._record_memory_history()` to start recording, `_dump_snapshot()` to write the file, and open it in memory_viz. Every allocation in the snapshot carries its size and the stack trace that made it, so any band of memory can be traced back to a line of code.
- **Healthy memory looks like a sawtooth:** there's a flat floor of weights and workspace. Each request climbs as its KV cache grows at 128 KB per token, so a 2,000-token request adds 2,000 × 128 KB ≈ 262 MB, and it falls back down when the request finishes.
- **A leak looks like a climbing floor:** every valley lands higher than the last, because something is never freed. The stack trace on the band that never frees names the line that is still holding the reference.
- **Fragmentation shows up as a gap between reserved and allocated:** the caching allocator reserves big segments and carves tensors out of them. Tensors with varying sizes and lifetimes chop those segments up, so the 20 GB is free but scattered, and one large request that needs contiguous space fails. That's how you can be out of memory with only 60 of 80 GB allocated.
- **The fixes:** set `expandable_segments:True`, use fixed-size pools (which is why vLLM pages its KV cache), and create fewer varying-size temporaries.

Key numbers: allocated = live tensors; reserved = held by the allocator; KV per token for Llama 3 8B = 128 KB.

Likely follow-up: "Why didn't a too-long request cause it?" A too-long request would fail the first time one arrived, not after 6 hours of running fine. On top of that, a real engine preallocates its KV cache up front, so the KV cache can't grow into an out-of-memory error.

> **TRAP**
>
> I guessed "the KV cache exploded" or "a request was over max length". Neither fits a crash after hours, and neither explains OOM with 20 GB free.

Q3 · A clean Nsight Systems trace of a live server PRIORITY

Lessons: 57

> **INTERVIEWER**
>
> You run `nsys profile` on a live vLLM server. The file is huge and mostly model loading, and during decode the GPU row shows one opaque block per step. How do you fix each problem, and what three things do you read off a clean decode window?

60-second answer

- **Fix the window:** `--delay=180 --duration=120` skips model loading and the cold first requests, so the capture records only steady traffic. `--kill=none` keeps the server alive after collection ends, so profiling doesn't take the service down.
- **Fix the opaque block:** each decode step is a replayed **CUDA graph**, which is why the GPU row shows one block per step. `--cuda-graph-trace=node` opens the graph up into its individual kernels, though it adds noticeable overhead. I'd also set `VLLM_WORKER_MULTIPROC_METHOD=spawn`, because nsys can't follow a forked worker.
- **Read 1, gaps:** holes in the GPU stream while users are waiting mean the device is idle, so it's not a kernel problem. If the API lane above is quiet, nothing was asked of the GPU; if it's busy, work was being issued but didn't arrive in time.
- **Read 2, pacing:** if the launch lane is packed but the kernels under it are thin slivers, the CPU is setting the pace. If the kernels sit shoulder to shoulder and are wider than their launch calls, the GPU is setting the pace.
- **Read 3, copies:** steady decode should keep the memory rows nearly silent. If there are wide or rhythmic copies between host and device, some state is crossing PCIe every step.

Key numbers: `--trace=cuda,nvtx,osrt`; NVTX ranges ("prefill", "decode") label phases on the timeline.

Likely follow-up: "When do you switch to Nsight Compute?" Only once the trace shows the GPU busy edge to edge and one named kernel owning the time, because only then is the problem inside a kernel rather than between kernels.

> **TRAP**
>
> I hadn't used Nsight Systems and couldn't name the window or CUDA graph flags, or the three readings.

Q4 · "Occupancy is terrible, retune the block size" PRIORITY

Lessons: 58, 61

> **INTERVIEWER**
>
> Nsight Compute on our decode GEMM shows a duration 3× longer than in production, memory throughput 92%, compute 2%, achieved occupancy 25%. A teammate wants to retune the block size. What do you tell them?

60-second answer

- **The 3× duration is an artifact of profiling.** `ncu` replays the launch once per metric group, 46 passes by default, and it restores memory between passes, serializes launches and pins clocks. So the duration says nothing about production, and only the metric values count.
- **92% memory and 2% compute is the healthy answer for this kernel.** A decode GEMM has an intensity of about 1 FLOP per byte, which is far below the H100 ridge of ≈ 295 FLOP per byte. So it should sit at the memory limit with compute idle, which means it's already at its floor.
- **Low occupancy doesn't convict a kernel on its own.** Extra warps only help by hiding latency, and with memory throughput already near its ceiling they'd have nothing more to fetch. Occupancy is worth worrying about only when it's low **and** both throughputs are low.
- **So retuning the block size buys nothing.** The only ways to speed this kernel up are fewer bytes, by quantizing the weights, or more tokens per weight read, by batching.
- **I'd predict the Speed Of Light numbers from the roofline before opening the report**, so that if the report disagrees with the prediction, the disagreement means something.

Key numbers: ridge = 989 TFLOPS ÷ 3.35 TB/s ≈ 295; decode GEMM I ≈ batch size; decode attention I ≈ 4 with GQA.

Likely follow-up: "What if the L2 hit rate on that GEMM is high?" A high hit rate means some of the bytes I counted never left the cache, so I'd redo the bound before trusting the report. I'd keep the scale in mind, though: 16 GB of weights can't loop in a 50 MB L2.

> **TRAP**
>
> I hadn't used Nsight Compute and couldn't read the report: replay time, the Speed Of Light pair and occupancy all needed explaining.

Q5 · Three elementwise ops in eager mode PRIORITY

Lessons: 60, 61

> **INTERVIEWER**
>
> In eager PyTorch a layer runs `x = x * scale`, `x = x + bias`, `x = gelu(x)` on a big BF16 activation. Why is it slow when the math is trivial? What does fusing it into one kernel change, which bucket does that target, and where do the intermediates live?

60-second answer

- **Every eager op starts and ends in HBM:** it reads x from HBM and writes x back. So three ops means **three round trips**, which is 6 tensor-sized transfers, all for a few FLOPs per element.
- **Each op on its own is memory-bound:** a simple op does 1 FLOP per element while reading 2 bytes and writing 2 bytes, so its intensity is $1 \div 4 \approx 0.25$ FLOP per byte. That's far below the ridge of ≈ 295, so each kernel is **memory-bound**.
- **Fusion does one read and one write in total:** that's 2 transfers instead of 6, so $\approx$ **3× less traffic** and $\approx 3\times$ faster. It targets the **memory** bucket, and it also helps overhead, since 3 launches become 1.
- **It's still memory-bound afterwards:** even with GELU's extra math, the fused kernel's intensity is only a few FLOP per byte, which is nowhere near 295. So fusion makes it faster, not compute-bound.
- **The intermediates stay in each thread's registers**, which take ≈ 1 cycle to access, with 256 KB per SM, so they never go back to HBM. Tiled ops like matmul and FlashAttention stage their data in **shared memory** instead.

Key numbers: 4096×4096 BF16 $= 33.5$ MB; one read + write $= 67$ MB $\div$ 3.35 TB/s $\approx 20\ \mu s$ against $0.25\ \mu s$ of math ($\approx 80\times$).

Likely follow-up: "When would fusion make a kernel compute-bound?" Only when the elementwise ops are fused into something that already reuses each byte hundreds of times, like a matmul epilogue. Elementwise chains on their own never reach the ridge, because each byte only feeds a few FLOPs.

> **TRAP**
>
> I said fusion moves the op to compute-bound. It cuts bytes, but the intensity stays tiny, so the kernel is still memory-bound, just faster.

Build it yourself

No exercise for this pack yet. The best practice is real: time a decode step with and without `torch.compile(mode="reduce-overhead")`, compute MBU for each, and capture one Nsight Systems window of each to see the gaps disappear.